Milestone 3
Stock Trend Prediction

Submitted by : Malli Nandhini (Team 5)

Course : Infosys Stock Trend AI Stock Market Forecasting Model – Batch 13

OBJECTIVE

- Complete model training
- Create dashboard for visualization
- Evaluate model performance

Model Training

- Dataset was divided into Training (80%) and Testing (20%) sets
- Training data was used to teach the model patterns
- Testing data was used to evaluate performance
- Two models were implemented: Linear Regression & Random Forest

```python
#PART 1 – Model Training

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Load dataset
data = pd.read_csv(r"C:\Users\Malli Mounika\Downloads\stock_data.csv")

# Use previous day's Close price to predict next day's Close
data['Next_Close'] = data['Close'].shift(-1)

# Remove last row (because next close will be null)
data = data.dropna()

# Features and Target
X = data[['Close']]
y = data['Next_Close']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

# Train Model
model = LinearRegression()
model.fit(X_train, y_train)
```

```python
# Train Model
model = LinearRegression()
model.fit(X_train, y_train)

print("Model Training Completed ")
```

```
Model Training Completed
```

```python
#PART 2 — Prediction & Evaluation

# Prediction
y_pred = model.predict(X_test)

# Metrics
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("="*50)
print("📊 STOCK TREND PREDICTION RESULTS")
print("="*50)
print("MAE :", round(mae,4))
print("RMSE :", round(rmse,4))
print("R2 Score :", round(r2,4))
print("="*50)
```

```
==================================================
📊 STOCK TREND PREDICTION RESULTS
==================================================
MAE : 0.5324
RMSE : 0.7351
R2 Score : 0.9965
==================================================
```

Linear Regression Model

- Linear Regression model was trained using training dataset.
- The model was used to predict stock closing prices.
- Why we use this model because, Linear Regression is a simple and widely used regression algorithm.
- It predicts continuous values such as stock prices.
- Helps understand the relationship between input features and target variable.
- Easy to implement and interpret.
- Works well when there is a linear relationship in data.

Random Forest Model

- Random Forest Regressor was trained using the training dataset.
- The model generated predictions for the test dataset.
- Performance was evaluated using R^2 Score, MAE, and RMSE.
- Helps in reducing overfitting compared to single decision tree.
- Captures complex and non-linear relationships in stock data.
- It builds multiple decision trees during training.

Evaluating Model

<u>R^2 Score (Coefficient of Determination)</u>

- Measures how well the model explains the variance in the target variable.
- Value closer to 1 indicates better performance.

<u>MAE (Mean Absolute Error)</u>

- Calculates the average absolute difference between actual and predicted values.
- Lower MAE indicates better accuracy.

<u>RMSE (Root Mean Squared Error)</u>

- Measures the square root of average squared differences between actual and predicted values.
- Lower RMSE indicates better model performance.

Historical Stock Graph Visualization

- Historical stock closing prices were visualized using a line graph to understand the trend and movement of the stock over time.
- X-axis represents Date
- Y-axis represents Closing Price
- The graph shows long-term growth and short-term fluctuations
- Helps identify upward, downward, or sideways trends

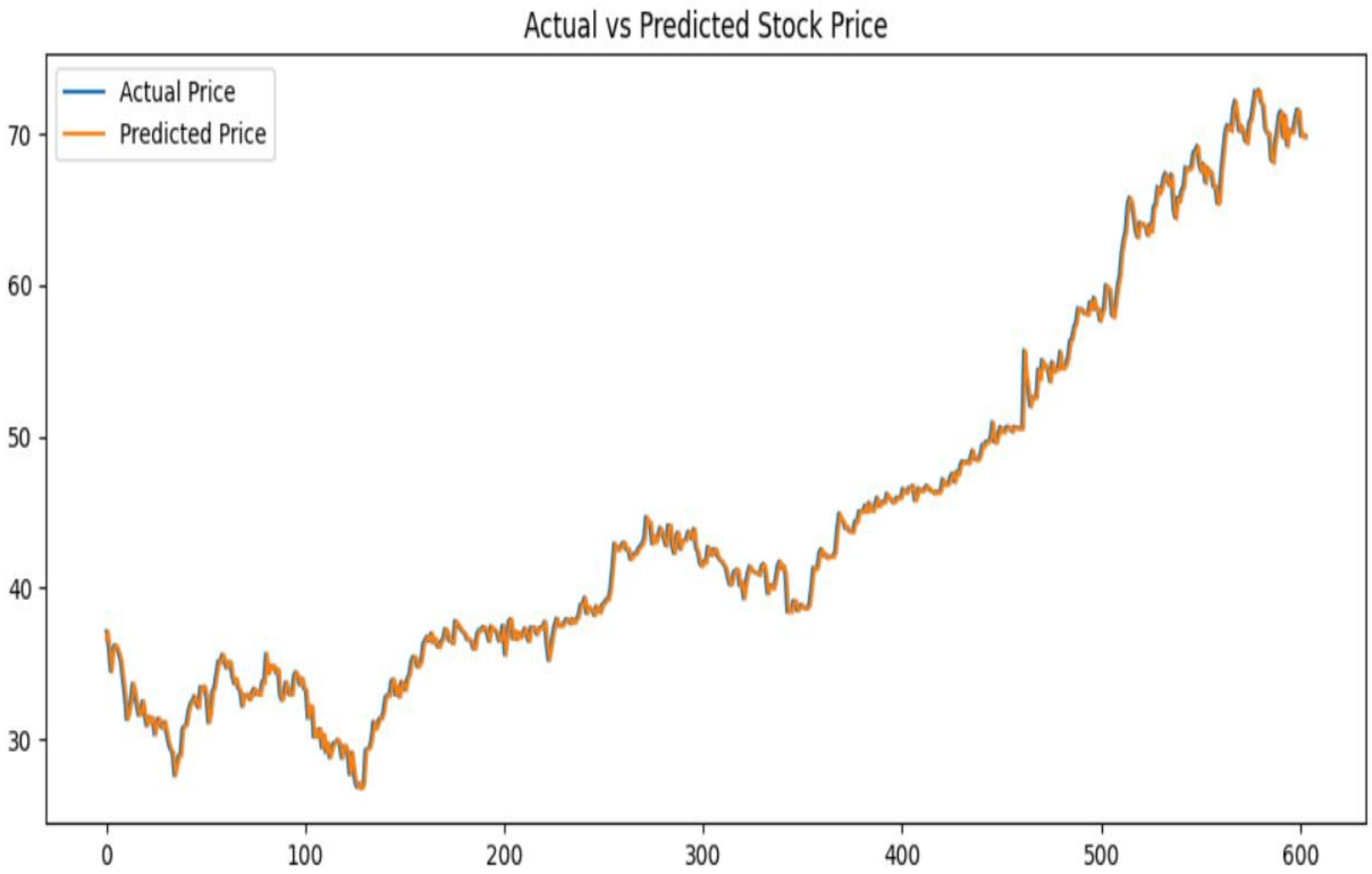
```
#PART 3 -  Historical Stock Graph

plt.figure(figsize=(12,5))
plt.plot(data['Close'])
plt.title(" Historical Stock Closing Price")
plt.xlabel("Days")
plt.ylabel("Price")
plt.show()
```

Actual vs Predicted Visualization

- After training the machine learning model, we compared the actual stock prices with predicted stock prices using a line graph.
- Blue line represents Actual Prices
- Orange line represents Predicted Prices
- X-axis represents Date/Test Data Points
- Y-axis represents Stock Price
- The closer the two lines, the better the model performance

```python
#PART 4 — 📊 Actual vs Predicted Graph

plt.figure(figsize=(12,5))
plt.plot(y_test.values, label="Actual Price")
plt.plot(y_pred, label="Predicted Price")
plt.title("Actual vs Predicted Stock Price")
plt.legend()
plt.show()
```

```
C:\Users\Malli Mounika\AppData\Local\Programs\Python\Python314\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Creating le
="best" can be slow with large amounts of data.
  fig.canvas.print_figure(bytes_io, **kw)
```


Actual vs Predicted Stock Price
Actual Price
Predicted Price
70
60
50
40
30
0
100
200
300
400
500
600

NEXT DAY PREDICTION

After training and evaluating the models, the trained model was used to predict the next day's stock closing price.

- The model takes input features such as Open, High, Low, and Volume
- Based on learned patterns from historical data
- The model generates the predicted closing price for the next trading day

```python
# PART 5 - Next Day Prediction


# Get last close price as DataFrame (with proper column name)
last_close_df = pd.DataFrame(
    [[data['Close'].iloc[-1]]],
    columns=['Close']
)


# Predict
next_day_prediction = model.predict(last_close_df)


print("Predicted Next Day Closing Price: ₹", round(next_day_prediction[0],2))
```

```
Predicted Next Day Closing Price: ₹ 69.72
```

Dashboard Creation

A visual dashboard was created to present the complete analysis and model results in a single view.

The dashboard includes:

- Historical Stock Price Graph
- Volume Traded
- Daily returns
- Actual vs Predicted Visualization
- Model Evaluation Metrics (MAE, RMSE, R^2)
- Model Comparison Table

📊📈 STOCK TREND PREDICTION DASHBOARD

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load dataset
df = pd.read_csv(r"C:\Users\Malli Mounika\Downloads\stock_data.csv")

# Convert Date column
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')

# Calculate Moving Average
df['MA50'] = df['Close'].rolling(window=50).mean()

# -------- DASHBOARD --------
plt.figure(figsize=(14,8))

# 1️⃣ Closing Price
plt.subplot(2,2,1)
plt.plot(df['Date'], df['Close'])
plt.title("Stock Closing Price")
plt.xticks(rotation=45)

# 2️⃣ Moving Average
plt.subplot(2,2,2)
plt.plot(df['Date'], df['Close'])
plt.plot(df['Date'], df['MA50'])
plt.title("50-Day Moving Average")
```

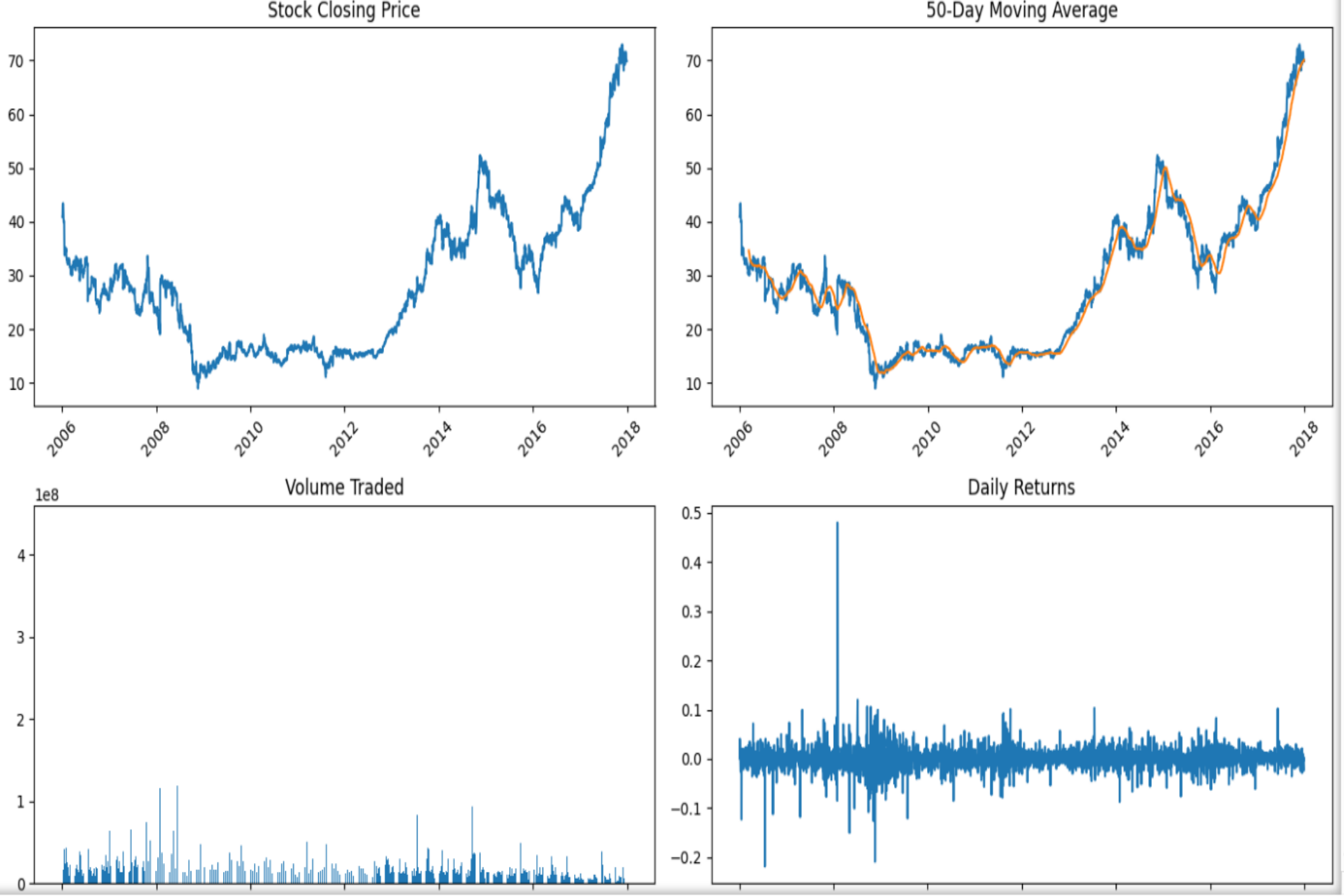
```python
# 2️⃣ Moving Average
plt.subplot(2,2,2)
plt.plot(df['Date'], df['Close'])
plt.plot(df['Date'], df['MA50'])
plt.title("50-Day Moving Average")
plt.xticks(rotation=45)

# 3️⃣ Volume
plt.subplot(2,2,3)
plt.bar(df['Date'], df['Volume'])
plt.title("Volume Traded")
plt.xticks(rotation=45)

# 4️⃣ Daily Returns
df['Returns'] = df['Close'].pct_change()
plt.subplot(2,2,4)
plt.plot(df['Date'], df['Returns'])
plt.title("Daily Returns")
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

y_pred = model.predict(X_test)

# Metrics
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

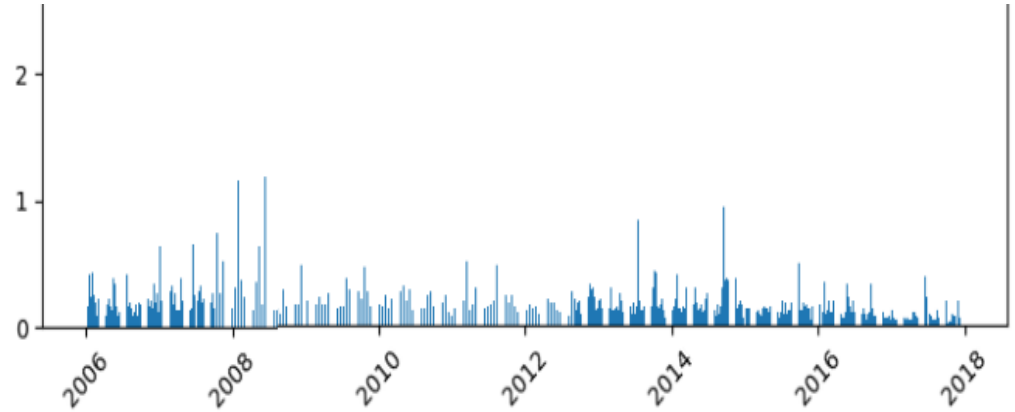
```python
# Convert to arrays for plotting
y_test_array = y_test.values
y_pred_array = y_pred.flatten()

# ---------------- DASHBOARD ----------------

print("="*60 + "\n")

# Next Day Prediction (No Warning Version)
last_close_df = pd.DataFrame(
    [[data['Close'].iloc[-1]]],
    columns=['Close']
)

next_day_prediction = model.predict(last_close_df)

print("\n🔮 Predicted Next Day Closing Price: ₹",
      round(next_day_prediction[0],2))
print("="*60 + "\n")

#Features and Target

# Use only Close price as feature
X = data[['Close']]   # IMPORTANT: Keep as DataFrame
y = data['Next_Close']
```

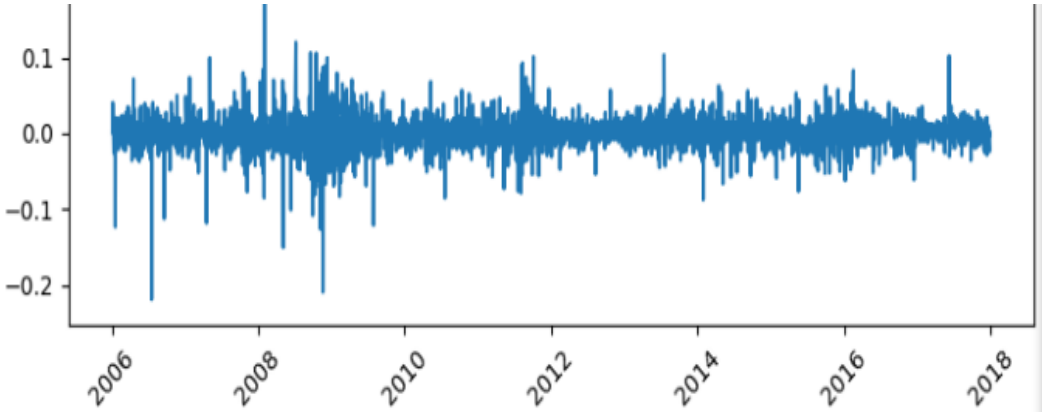
```python
# Split (No shuffle for time series)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, shuffle=False
)
# LINEAR REGRESSION MODEL
# ◆ Train Model

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

print("Linear Regression Training Completed")
# ◆ Prediction

linear_pred = linear_model.predict(X_test)
# ◆ Evaluation

linear_mae = mean_absolute_error(y_test, linear_pred)
linear_rmse = np.sqrt(mean_squared_error(y_test, linear_pred))
linear_r2 = r2_score(y_test, linear_pred)

print("\n ◆ Linear Regression Results")
print("MAE :", round(linear_mae,4))
print("RMSE :", round(linear_rmse,4))
print("R2 Score :", round(linear_r2,4))
print("="*60 + "\n")
```

```python
#RANDOM FOREST MODEL
# 🔹 Train Model
from sklearn.ensemble import RandomForestRegressor
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train, y_train)

print("🔹 Random Forest Training Completed")
# 🔹 Prediction

rf_pred = rf_model.predict(X_test)
# 🔹 Evaluation

rf_mae = mean_absolute_error(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))
rf_r2 = r2_score(y_test, rf_pred)

print("\nRandom Forest Results")
print("MAE :", round(rf_mae,4))
print("RMSE :", round(rf_rmse,4))
print("R2 Score :", round(rf_r2,4))
print("="*60 + "\n")
```

```python
#Comparison Graph
plt.figure(figsize=(12,5))

plt.plot(y_test.values, label="Actual")
plt.plot(linear_pred, label="Linear Regression")
plt.plot(rf_pred, label="Random Forest")

plt.title("Model Comparison")
plt.legend()
plt.grid()
plt.show()
```

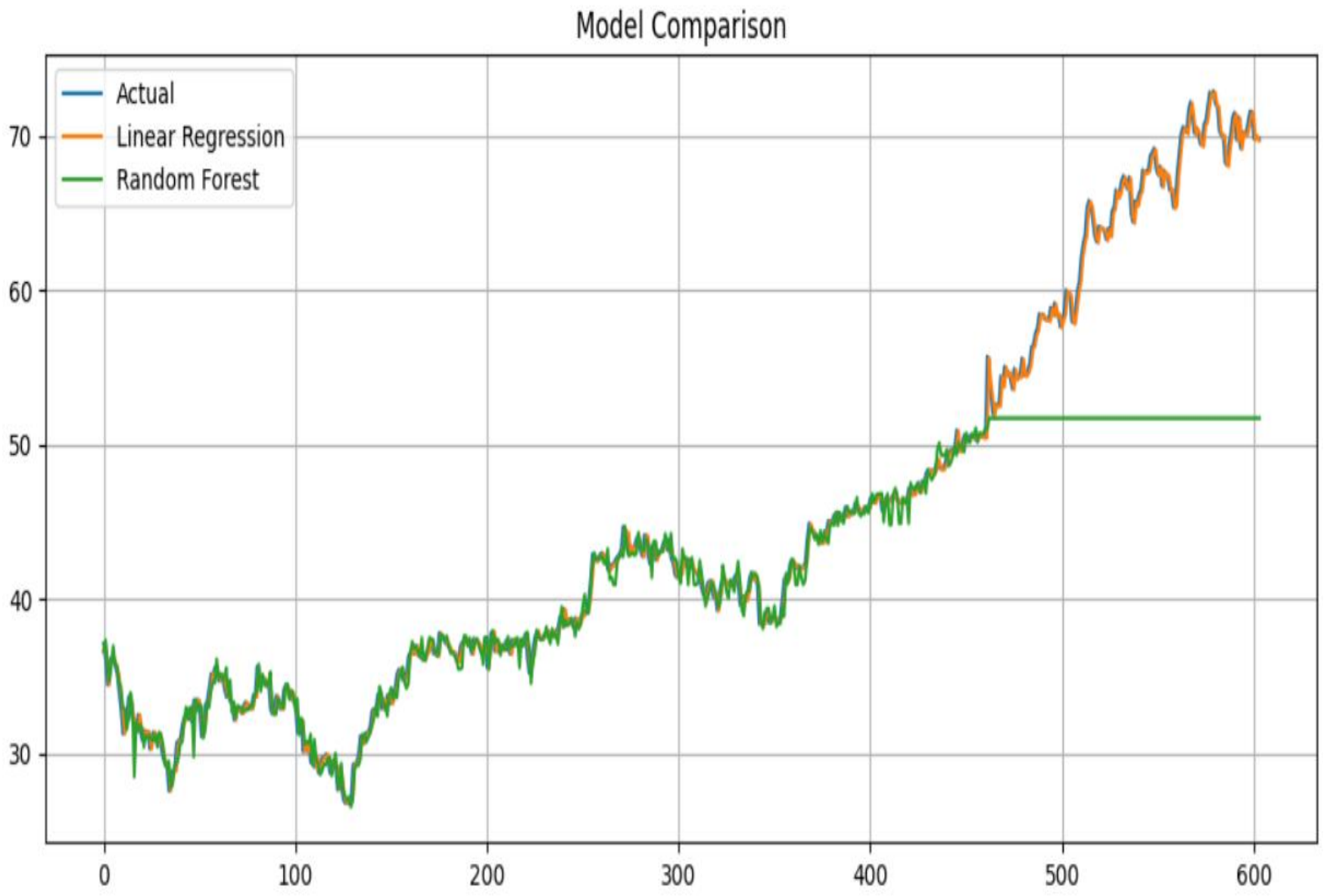

Stock Closing Price
50-Day Moving Average
Volume Traded
Daily Returns
1e8
70
60
50
40
30
20
10
4
3
2
1
0
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
2006
2008
2010
2012
2014
2016
2018






```
============================================================


🔮 Predicted Next Day Closing Price: ₹ 69.72
============================================================

Linear Regression Training Completed

🔹 Linear Regression Results
MAE : 0.5324
RMSE : 0.7351
R2 Score : 0.9965
============================================================

🔹 Random Forest Training Completed

Random Forest Results
MAE : 3.3958
RMSE : 6.7472
R2 Score : 0.7014
============================================================
```

==========


Model Comparison
Actual
Linear Regression
Random Forest
70
60
50
40
30
0
100
200
300
400
500
600

CONCLUSION

- This project successfully implemented stock price prediction using Machine Learning techniques.
- Both Linear Regression and Random Forest models were trained and evaluated using MAE, RMSE, and R^2 Score.
- The dashboard provided a structured visualization of historical trends and model comparison, completing the objectives effectively.